

Fast Multiplication in Canonical Nim Coordinates

GPT 5.6 Sol

a9lim

Abstract

For $n = 2^k$, the finite nimbers below 2^n form $\mathbb{F}_{2^n}$, but their literal n -bit words are not an arbitrary polynomial basis. They are the multivariate basis of the Conway Artin–Schreier tower generated by $c_i = 2^{2^i}$ with

$$c_i^2 + c_i = \prod_{j < i} c_j.$$

We give explicit forward and inverse transforms from these canonical words to the primitive Artin–Schreier tower of De Feo and Schost. The specialization has a particularly simple form: the image of each canonical generator is an affine shift $z_i + r_i$ of the primitive generator, and the two coefficient blocks change by

$$(A, B) \mapsto (A + Br_i, B).$$

The shift is its own inverse in characteristic two.

After uniform preprocessing, the resulting multiplication algorithm uses

$$O(M(n) \log n + n \log^3 n)$$

bit operations and $n \log^{O(1)} n$ space, where $M(n)$ is binary-polynomial multiplication cost. Pre-computation is also quasi-linear up to logarithmic factors and stores no dense change-of-basis matrix or multiplication tensor. The inverse transform returns the original tower coefficients, whose ordering is exactly the literal nim-word ordering; the output is therefore the canonical nim product, not merely an element of an abstract isomorphic field.

This closes the mathematical transform and complexity part of the problem. An arbitrary-width implementation, differential tests, and crossover measurements remain engineering work. Lean checks the affine Artin–Schreier identity, the involutive block transform, preservation of the tower equations, and exact multiplication transport; the asymptotic algorithms and the concrete Conway identification remain paper-level and cited mathematics.

1 Statement of the result

Nim addition and multiplication make the ordinals a characteristic-two field-like proper class; the finite segment below 2^{2^k} is the field $\mathbb{F}_{2^{2^k}}$ [1, 4]. Put

$$n = 2^k, \quad K_k = \{0, \dots, 2^n - 1\} = \mathbb{F}_{2^n}.$$

An element of K_k is supplied as its literal n -bit integer word. The problem is to multiply in time quasi-linear in this representation length, without replacing the canonical word by an unspecified field presentation or hiding a quadratic table in preprocessing.

We use a standard multiplication function M satisfying

$$M(m) \geq m, \quad \frac{M(m)}{m} \text{ is nondecreasing,} \quad M(2m) = O(M(m)).$$

We also choose the usual fast polynomial multiplication, remainder, and transposed algorithms with $m \log^{O(1)} m$ workspace. These are the same regularity assumptions used in the fast-tower analysis [2].

Theorem 1.1 (canonical-word multiplication). *For every $k \geq 0$, let $n = 2^k$. There is a uniform preprocessing algorithm which constructs $n \log^{O(1)} n$ bits of level data, followed by a multiplication algorithm*

$$\text{mul}_k : K_k \times K_k \longrightarrow K_k$$

on literal canonical nim words. Its online cost is

$$O(kM(n) + nk^3) = O(M(n) \log^{O(1)} n),$$

and its auxiliary space is $n \log^{O(1)} n$ bits. The preprocessing cost is

$$O(k^2 M(n) + nk^4 + M(2n)).$$

No dense basis-conversion matrix or multiplication tensor is stored.

The proof has three parts. First, the literal bit ordering is identified with the monomial ordering of one explicit quadratic tower. Second, that tower is mapped to the primitive tower of De Feo and Schost by recursively computed affine generator shifts. Third, the published level-embedding bounds are specialized and summed without suppressing conversion or preprocessing.

2 The literal word is the Conway tower basis

It is useful to index generators from one. Set

$$u_i = c_{i-1} = 2^{2^{i-1}} \quad (i \geq 1), \quad A_1 = 1, \quad A_i = \prod_{j < i} u_j \quad (i \geq 2).$$

The canonical finite-nimber identities are

$$u_i^2 + u_i = A_i. \tag{1}$$

Equivalently, $c_i^2 = c_i + \prod_{j < i} c_j$. This is the finite Conway–Lenstra tower relation, not an arbitrary choice of irreducible polynomials [1, 4].

Define the presented tower

$$T_0 = \mathbb{F}_2, \quad T_i = T_{i-1}[U_i]/(U_i^2 + U_i + A_i),$$

where after adjoining u_i we put $A_{i+1} = A_i u_i$.

Proposition 2.1 (trace-one sources). *For every $i \geq 1$,*

$$\text{Tr}_{T_{i-1}/\mathbb{F}_2}(A_i) = 1.$$

Consequently $X^2 + X + A_i$ is irreducible over T_{i-1} , $[T_i : T_{i-1}] = 2$, and the 2^i monomials

$$\prod_{j=1}^i u_j^{\epsilon_j}, \quad \epsilon_j \in \{0, 1\},$$

form an $\mathbb{F}_2$ -basis of T_i .

Proof. We prove simultaneously that every T_i is a field and that the next source has trace one. The initial field is $T_0 = \mathbb{F}_2$ and $A_1 = 1$. Suppose T_{i-1} is a field and the trace of A_i is one. The finite-field Artin–Schreier criterion makes $X^2 + X + A_i$ irreducible, so T_i is a quadratic field extension. Its two conjugates of u_i are u_i and $u_i + 1$, whence

$$\mathrm{Tr}_{T_i/T_{i-1}}(u_i) = 1.$$

It follows that

$$\mathrm{Tr}_{T_i/T_{i-1}}(A_{i+1}) = \mathrm{Tr}_{T_i/T_{i-1}}(A_i u_i) = A_i.$$

Trace transitivity proves the displayed trace identity at level $i + 1$. This completes the induction by trace transitivity. The criterion used above is the standard equivalence between irreducibility of $X^2 + X + a$ and absolute trace one [5]. Every step therefore doubles the degree, and the stated monomials are the resulting iterated quadratic basis. $\square$

Proposition 2.2 (literal ordering). *Let $0 \leq e < 2^k$ and write*

$$e = \sum_{i=1}^k \epsilon_i 2^{i-1}, \quad \epsilon_i \in \{0, 1\}.$$

Then the number represented by the lone word bit in position e is

$$2^e = \prod_{i=1}^k u_i^{\epsilon_i}$$

under nim multiplication. Hence, for a word $x = \sum_{e < n} x_e 2^e$, the bit vector (x_e) is exactly the coordinate vector of x on the monomial basis of Proposition 2.1, in ordinary increasing bit order.

Proof. Distinct Fermat powers nim-multiply as their ordinary product, while nim addition of finite numbers is bitwise exclusive-or [1, 4]. Since $u_i = 2^{2^{i-1}}$, the displayed product is $2^{\sum_i \epsilon_i 2^{i-1}} = 2^e$. Distributing over the set bits of x gives the coordinate statement. $\square$

This proposition is the representation boundary that an abstract finite-field isomorphism would not supply. From now on, splitting an $N_i = 2^i$ bit canonical word into its low and high halves means writing

$$x = x_0 + x_1 u_i, \quad x_0, x_1 \in T_{i-1}. \quad (2)$$

3 The fast primitive tower

We specialize the primitive Artin–Schreier tower of De Feo and Schost to $p = 2$, base degree $d = 1$, and $Q_0 = X + 1$ [2]. Let $V_0 = \mathbb{F}_2$ and

$$V_i = V_{i-1}(z_i), \quad z_i^2 + z_i = g_i,$$

where

$$g_1 = 1, \quad g_2 = z_1, \quad g_i = z_{i-1}^3 \quad (i \geq 3). \quad (3)$$

Their primitive-tower theorem says that z_i generates all of V_i over $\mathbb{F}_2$. If Q_i is its minimal polynomial, then

$$V_i = \mathbb{F}_2[Z]/(Q_i(Z))$$

with the univariate power basis

$$1, z_i, \dots, z_i^{N_i-1}.$$

Multiplication and reduction in this basis cost $O(M(N_i))$ bit operations.

The same work gives mutually inverse level maps

$$\text{Push}_i : V_i \longrightarrow V_{i-1} \oplus V_{i-1}z_i, \quad \text{Lift}_i = \text{Push}_i^{-1},$$

with cost

$$L(i) = O(N_i t^2 + M(N_i)). \quad (4)$$

They are implemented by divide-and-conquer reduction, transposed reduction, and trace parameterization; they are algorithms, not stored matrices. The construction also computes the Q_i , modular-reduction data, traces, and pseudotraces used by the level maps and the fast Artin–Schreier solver.

The primitive tower is not the canonical nim tower. The first two sources in the canonical and primitive displays coincide, but at the third step the primitive source is z_2^3 , whereas the canonical source is the product of the first two generators. The conversion below is therefore essential.

4 Affine-shift isomorphisms

Write

$$\text{AS}(x) = x^2 + x.$$

In characteristic two,

$$\text{AS}(x + y) = \text{AS}(x) + \text{AS}(y). \quad (5)$$

We now construct compatible field isomorphisms

$$\sigma_i : T_i \longrightarrow V_i$$

and expose their exact coordinate action.

Theorem 4.1 (selected affine shifts). *Set $\sigma_0 = \text{id}_{\mathbb{F}_2}$. Suppose σ_{i-1} has been constructed, and put*

$$S_i = \sigma_{i-1}(A_i) = \prod_{j < i} s_j, \quad s_j = \sigma_j(u_j).$$

There is an $r_i \in V_{i-1}$ satisfying

$$r_i^2 + r_i = S_i + g_i. \quad (6)$$

For any such r_i , the assignment

$$s_i = z_i + r_i \quad (7)$$

extends σ_{i-1} uniquely to a field isomorphism $\sigma_i : T_i \rightarrow V_i$. The shifts r_i are computable uniformly by the fast Artin–Schreier solver.

Proof. Proposition 2.1 and invariance of trace under field isomorphism give

$$\text{Tr}_{V_{i-1}/\mathbb{F}_2}(S_i) = 1.$$

The primitive-tower theorem gives the same identity for g_i . Therefore $S_i + g_i$ has trace zero. The image of $x \mapsto x^2 + x$ on a finite characteristic-two field is exactly the absolute-trace kernel, so the displayed shift equation has a solution. De Feo and Schost give a recursive quasi-linear algorithm for finding one.

The primitive relation, shift equation, and additivity of AS yield

$$s_i^2 + s_i = g_i + (S_i + g_i) = S_i.$$

Thus s_i satisfies the transported defining equation of u_i . The quotient universal property extends σ_{i-1} to a homomorphism $T_i \rightarrow V_i$. Both fields have 2^{N_i} elements, and a unital field homomorphism is injective, so it is an isomorphism. $\square$

The affine form is stronger than merely knowing some coordinates for s_i . Indeed, any root in V_i of an Artin–Schreier polynomial with source in V_{i-1} is either downstairs or has z_i -coefficient one. The selected root is not downstairs because its source has trace one there. Hence the affine formula $s_i = z_i + r_i$ loses no generality and needs no stored coefficient inverse.

Remark 4.2 (first nontrivial shift). Let $a = z_1$, $b = z_2$, and $t = z_3$. Then

$$a^2 + a = 1, \quad b^2 + b = a, \quad t^2 + t = b^3 = ab + a + b.$$

Put $r_3 = ab + a + b$. Direct reduction gives $r_3^2 + r_3 = a + b$, and hence

$$(t + r_3)^2 + (t + r_3) = ab,$$

the canonical third source. This is where the two towers first diverge.

5 Forward and inverse transforms

For $A, B \in V_{i-1}$, the entire level conversion rests on

$$A + B(z_i + r_i) = (A + Br_i) + Bz_i. \tag{8}$$

The coefficient map

$$\tau_i(A, B) = (A + Br_i, B) \tag{9}$$

is an involution, since

$$(A + Br_i) + Br_i = A.$$

Define the forward transform F_i recursively. Set F_0 to the identity. Given the canonical low–high split, compute

$$\begin{aligned} X_0 &= F_{i-1}(x_0), & X_1 &= F_{i-1}(x_1), \\ F_i(x) &= \text{Lift}_i(X_0 + r_i X_1, X_1). \end{aligned} \tag{10}$$

Thus $F_i(x)$ is returned in the univariate power basis of z_i .

Define the inverse G_i in the opposite direction. Set G_0 to the identity. For $y \in V_i$, compute

$$\text{Push}_i(y) = (Y_0, Y_1),$$

and put

$$G_i(y) = G_{i-1}(Y_0 + r_i Y_1) + u_i G_{i-1}(Y_1). \tag{11}$$

The right side is emitted as the low and high halves of a canonical word; it does not require a tower multiplication.

Proposition 5.1 (transform correctness). *For every i , F_i is the coordinate algorithm for σ_i , G_i is the coordinate algorithm for σ_i^{-1} , and*

$$G_i F_i = \text{id}_{T_i}, \quad F_i G_i = \text{id}_{V_i}.$$

Proof. Assume the statement at level $i - 1$. Replacing u_i by $s_i = z_i + r_i$ in $x_0 + x_1 u_i$ gives the forward formula by the block-shift identity, so F_i computes σ_i . Conversely, the involutive block map rewrites the pushed-down coefficients of y from the z_i basis to the s_i basis, after which the inverse formula applies the lower inverse transforms. Since τ_i is involutive and the level maps $\text{Push}_i, \text{Lift}_i$ are inverse, both displayed identities follow by induction. $\square$

We may now define

$$\text{mul}_k(x, y) = G_k(F_k(x)F_k(y) \bmod Q_k). \quad (12)$$

Corollary 5.2 (exact canonical output). *The bit string returned by the displayed multiplication algorithm is the literal canonical nim product of x and y .*

Proof. Proposition 5.1 says that F_k and G_k are inverse field isomorphisms, so decoding the returned vector in T_k gives the field product of the decoded inputs. Proposition 2.2 says that the T_k basis coordinates, in the order returned by G_k , are exactly the literal nim-word bits. Uniqueness of coordinates in that basis proves the claim. $\square$

6 Time and space bounds

Let $T(i)$ be the cost of either transform at level i . A depth-first implementation makes two recursive calls, one level conversion, and at most one multiplication in V_{i-1} . By the level-embedding bound,

$$T(i) \leq 2T(i-1) + O(L(i) + M(N_{i-1})). \quad (13)$$

Unrolling at $n = N_k$ gives

$$T(k) = O\left(\sum_{i=1}^k 2^{k-i}(N_i i^2 + M(N_i))\right).$$

The first summand at depth i is ni^2 , so its total is $O(nk^3)$. Monotonicity of $M(m)/m$ gives

$$2^{k-i}M(2^i) \leq M(2^k) = M(n),$$

so the multiplication contribution is $O(kM(n))$. Hence

$$T(k) = O(nk^3 + kM(n)). \quad (14)$$

Two forward transforms, one $O(M(n))$ univariate multiplication, and one inverse transform have the same asymptotic bound. This proves the online part of Theorem 1.1.

For completeness, the preprocessing is not free. In the notation of De Feo and Schost, the level-embedding cost is $L(i)$ and their pseudotrace bound specializes at $p = 2, d = 1$ to

$$\text{PT}(i) = O(i^2 L(i)).$$

Their Artin–Schreier solver has the same dominant bound. Constructing the primitive tower, its remainder and trace data, and all shifts through level k therefore costs

$$O(k^2 L(k) + M(2n)) = O(k^2 M(n) + nk^4 + M(2n)).$$

This is quasi-linear up to logarithms. In particular, no quadratic-time preprocessing is being exchanged for fast online multiplication.

The stored primitive polynomials, reciprocal data, trace series, selected shifts, and level maps have total size $n \log^{O(1)} n$. More explicitly, the fast-tower construction stores $O(k^2)$ tower elements; charging every one the top-level width gives the conservative bound $O(nk^2)$ bits. The actual level-weighted sum is smaller for data kept only at their construction level. Push-down and lift-up are divide-and-conquer algorithms, not N_i -square matrices. Running the two recursive children sequentially retains only a geometric stack of coefficient blocks, and the selected polynomial routines use quasi-linear workspace. Thus both working storage and conversion data are $n \log^{O(1)} n$.

7 Formal and implementation boundary

The Lean module `Ogdoad.NimFastMultiplication` checks the following algebraic joins used above:

- the finite-field equivalence between trace one and irreducibility of $X^2 + X + a$;
- the trace carry through a quadratic generator and the direct three-variable-product quadratic multiplication split;
- the shifted-source identity of Theorem 4.1;
- the involutivity and evaluation identity for $(A, B) \mapsto (A + Br, B)$;
- preservation of the canonical tower equations under a ring equivalence; and
- exact recovery of basis coordinates after multiplication is transported through a ring equivalence.

The existing finite-field development also identifies the Artin–Schreier image with the trace kernel. These declarations are kernel-checked by Lean and Mathlib [3, 6].

Lean does not formalize the concrete class of finite nimbers, the Conway–Lenstra identities, the De Feo–Schost divide-and-conquer algorithms, or their asymptotic analysis. Those are respectively paper-level cited mathematics and external algorithmic theorems. The formal module therefore checks the load-bearing algebraic specialization without pretending to be an end-to-end verification of Theorem 1.1.

Theorem 1.1 and Corollary 5.2 supply the forward and inverse transforms, their correctness, multiplication, preprocessing, time, and space bounds. The mathematical portion of fast multiplication in canonical nim coordinates is consequently closed. What remains is to implement arbitrary-width bit vectors and polynomial arithmetic, compare exhaustively with the mex oracle on small fields, compare differentially with the current fixed-width backend, and measure the crossover at which the fast path should replace it.

References

- [1] John H. Conway. *On Numbers and Games*. Academic Press, London, 1976.
- [2] Luca De Feo and Éric Schost. Fast arithmetics in Artin–Schreier towers over finite fields. *Journal of Symbolic Computation*, 47(7):771–792, 2012. doi: 10.1016/j.jsc.2011.12.008. URL <https://arxiv.org/abs/1002.2594>.
- [3] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In André Platzer and Geoff Sutcliffe, editors, *Automated Deduction—CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.
- [4] H. W. Lenstra, Jr. Nim multiplication. *Séminaire de Théorie des Nombres de Bordeaux*, 7, 1977–1978. Exposé 11, pp. 1–24.
- [5] Rudolf Lidl and Harald Niederreiter. *Finite Fields*, volume 20 of *Encyclopedia of Mathematics and its Applications*. Cambridge University Press, 2nd edition, 1997.
- [6] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381. Association for Computing Machinery, 2020. doi: 10.1145/3372885.3373824.